

Chaîne de compilation pour Modane

Création de fonctions optimisées avec MLIR

Louis-Max Harter

2026-06-22

France

Modane: Simplifier l'écriture de code

- Syntaxe haut niveau
- On décrit les opérations

```
@ComputeLoop
computeFjr
  in p,
  in C,
  in Ajr,
  in uj,
  in ur,
  out F {
    for (j in cells())
      for (r in nodesOfCell(j))
        F{j,r} = p{j} * C{j,r}
          + prodTensVec(Ajr{j,r}, (uj{j} -
ur{r}));
  }
```

Listing 1: Modane

Modane: Simplifier l'écriture de code

- Syntaxe haut niveau
- On décrit les opérations

```
@ComputeLoop
computeFjr
  in p,
  in C,
  in Ajr,
  in uj,
  in ur,
  out F {
  for (j in cells())
    for (r in nodesOfCell(j))
      F{j,r} = p{j} * C{j,r}
      + prodTensVec(Ajr{j,r}, (uj{j} -
ur{r}));
}
```

Listing 3: Modane

```
void computeFjr() {
  ENUMERATE_(Cell, j, defaultMesh()->allCells()) {
    NodeConnectedListView tmp_r_container = (*j).nodes();
    for(int r=0, nb_r=tmp_r_container.size(); r<nb_r; ++r) {
      m_F(j, r) = m_p[j] * m_C(j, r)
      + Arcane::math::prodTensVec(
        m_Ajr(j, r),
        (m_uj[j] - m_ur[tmp_r_container[r]]
        )
      );
    }
  }
}
```

Listing 4: Équivalent Arcane

- On travaille avec des boucles
- On souhaite améliorer les performances
- Quelles optimisations sont possibles ?
- Est-ce que -O3 suffira ?!

- On travaille avec des boucles
- On souhaite améliorer les performances
- Quelles optimisations sont possibles ?
- Est-ce que -O3 suffira ?!

```
int main(){  
    int somme = 0;  
    int n = ??; /* Connue à l'exécution seulement */  
  
    /* somme de 0 à n-1 */  
    for(int i = 0; i < n; i++){  
        somme += i;  
    }  
  
    /* somme de n à 1 */  
    for(int i = 0; i < n; i++){  
        somme += n - i;  
    }  
  
    /* somme = n * n */  
    std::cout << somme << std::endl;  
}
```

Listing 6: C++, deux boucles classiques

LLVM vous avez dit ?

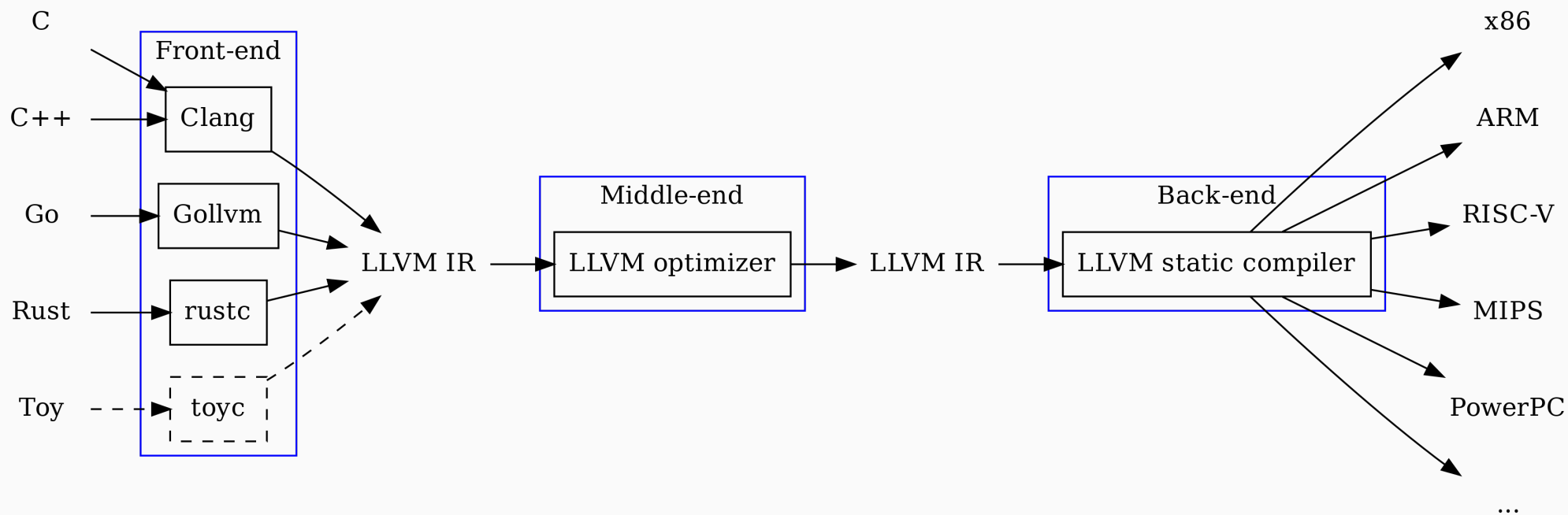


Figure 1: LLVM pour la compilation

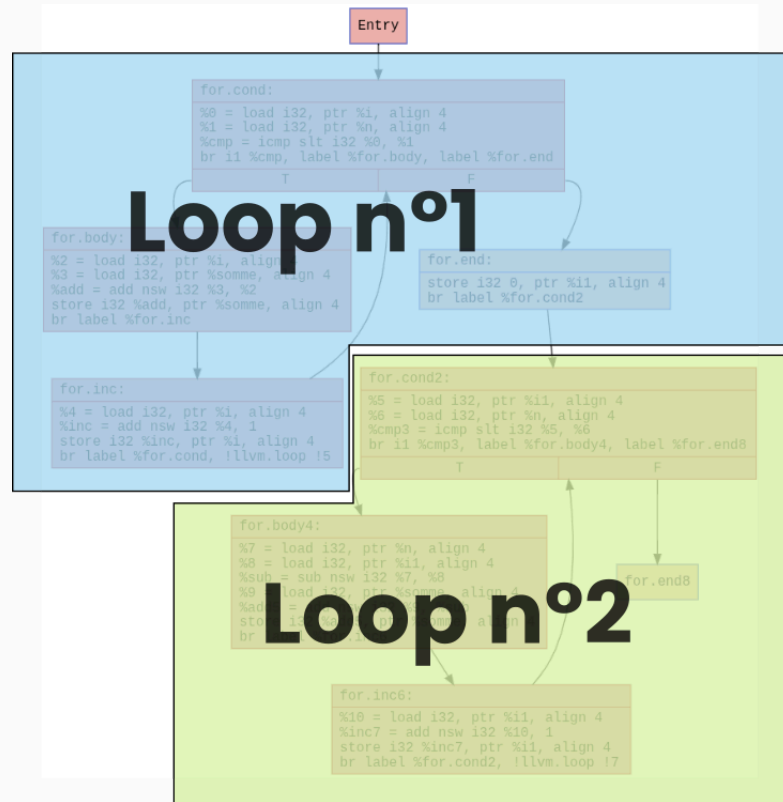


Figure 2: Résultat attendu

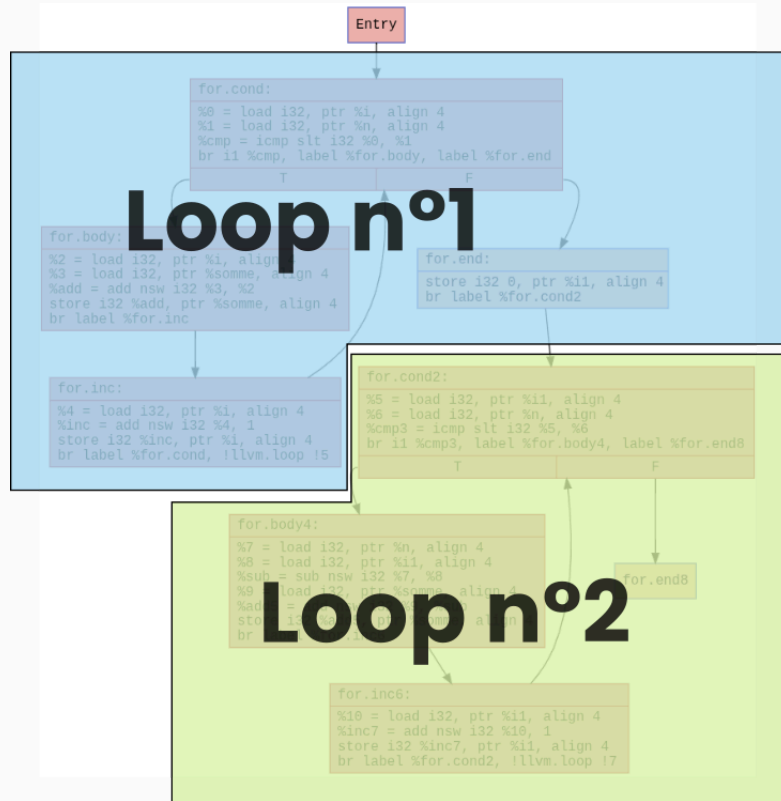


Figure 2: Résultat attendu

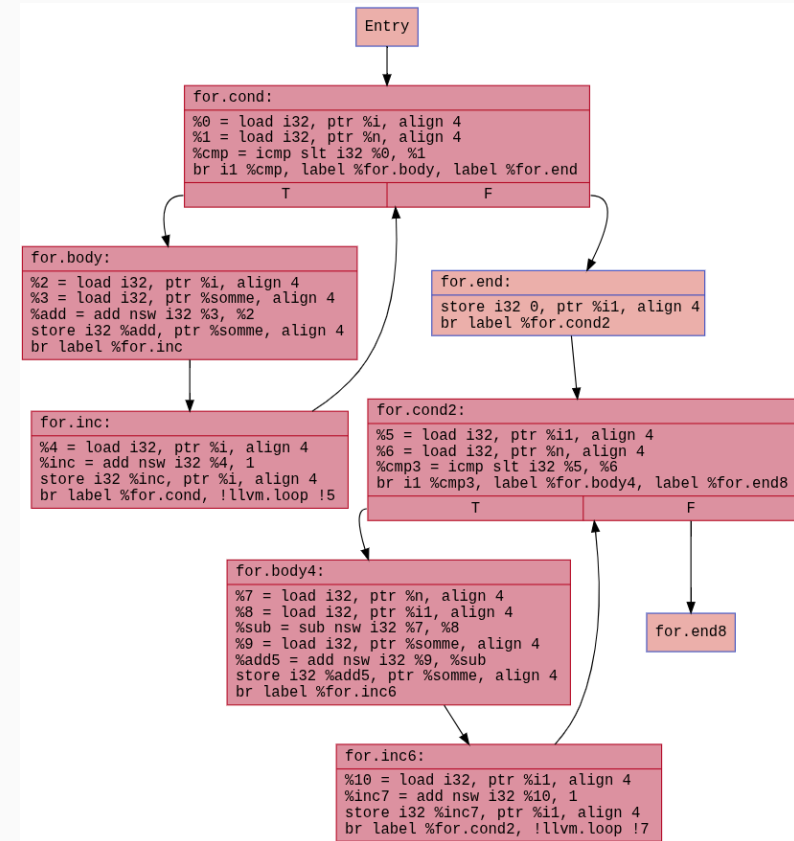


Figure 3: Résultat obtenu

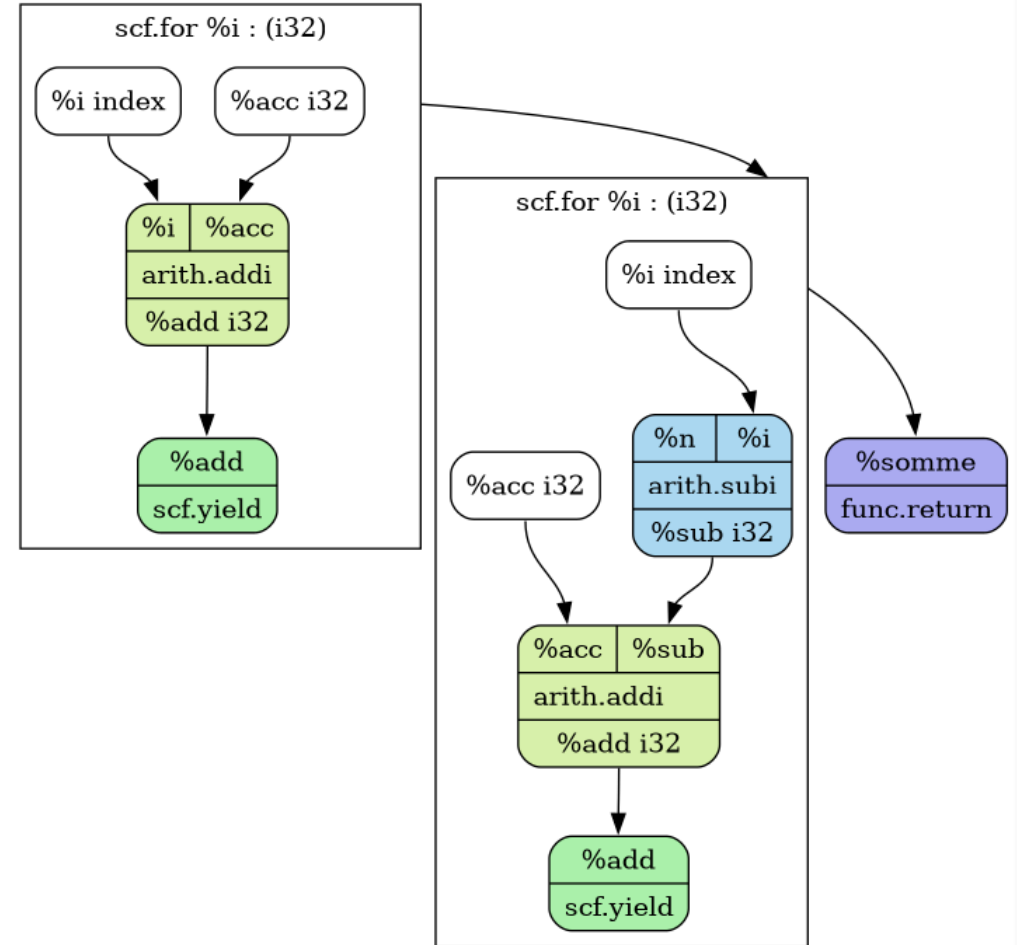


Figure 4: Résultat obtenu

MLIR: Multi-Level Intermediate Representation

- Plus haut niveau d'abstraction
- Différents dialectes (LineAlg, Affine, GPU, ...)
- Des optimisations de différents niveaux
 - loop-hoist-memref
 - stencil-unroll
 - stencil-inlining
 - ...
- On descend ensuite en LLVM

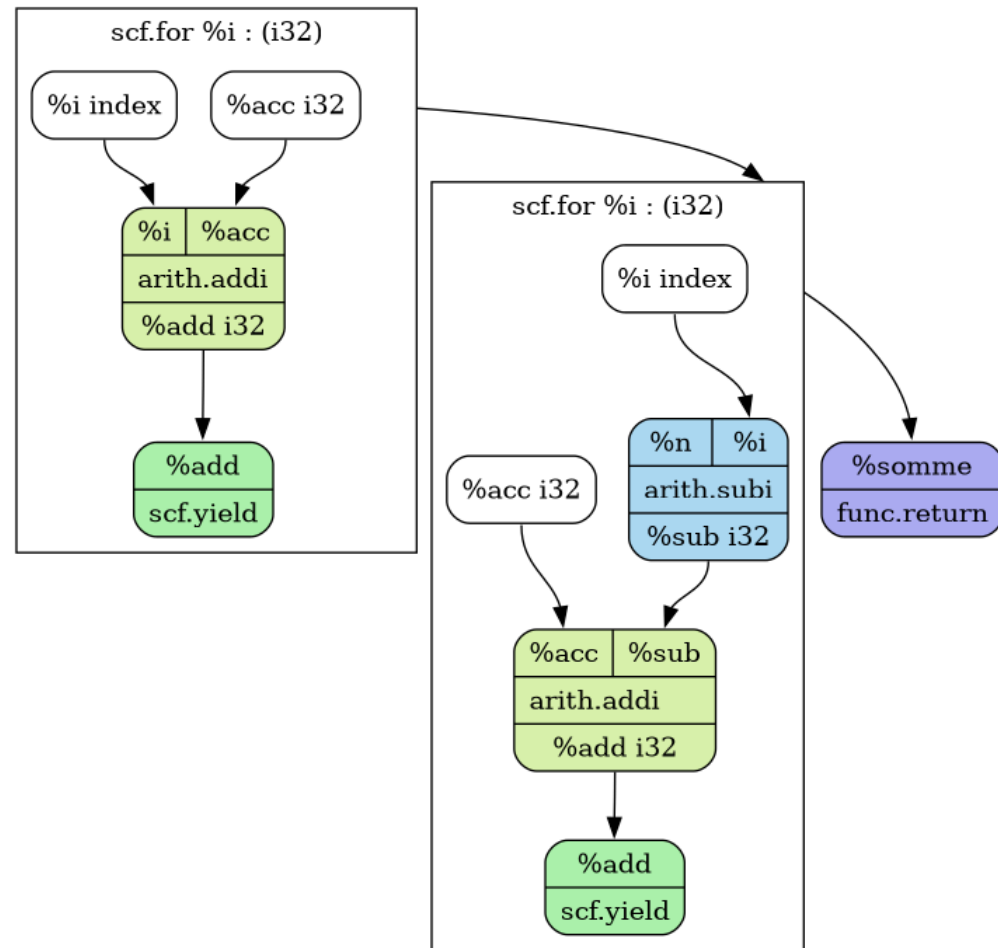


Figure 4: Résultat obtenu

Petit récapitulatif

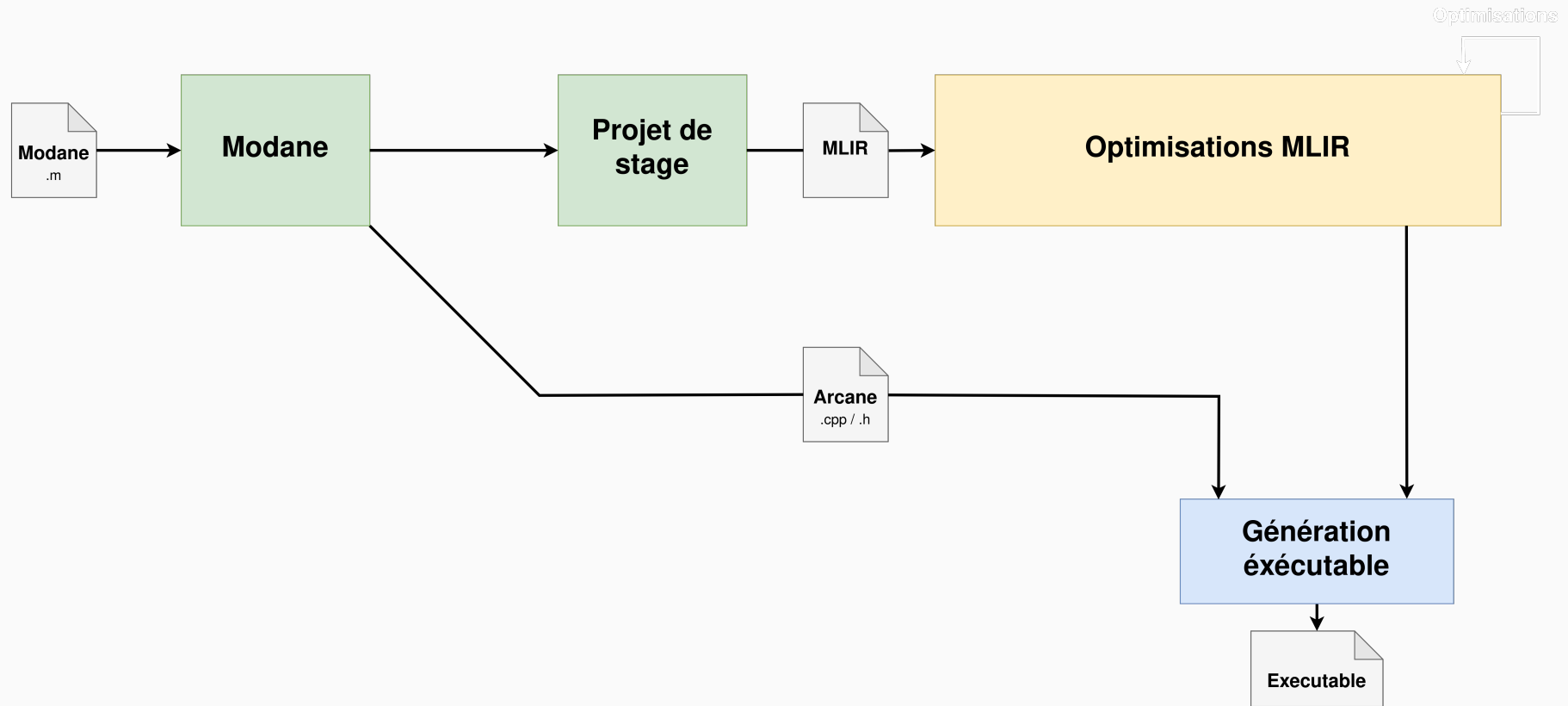


Figure 5: Organisation de la chaine de compilation (simplifiée)

Avec plus de détails !

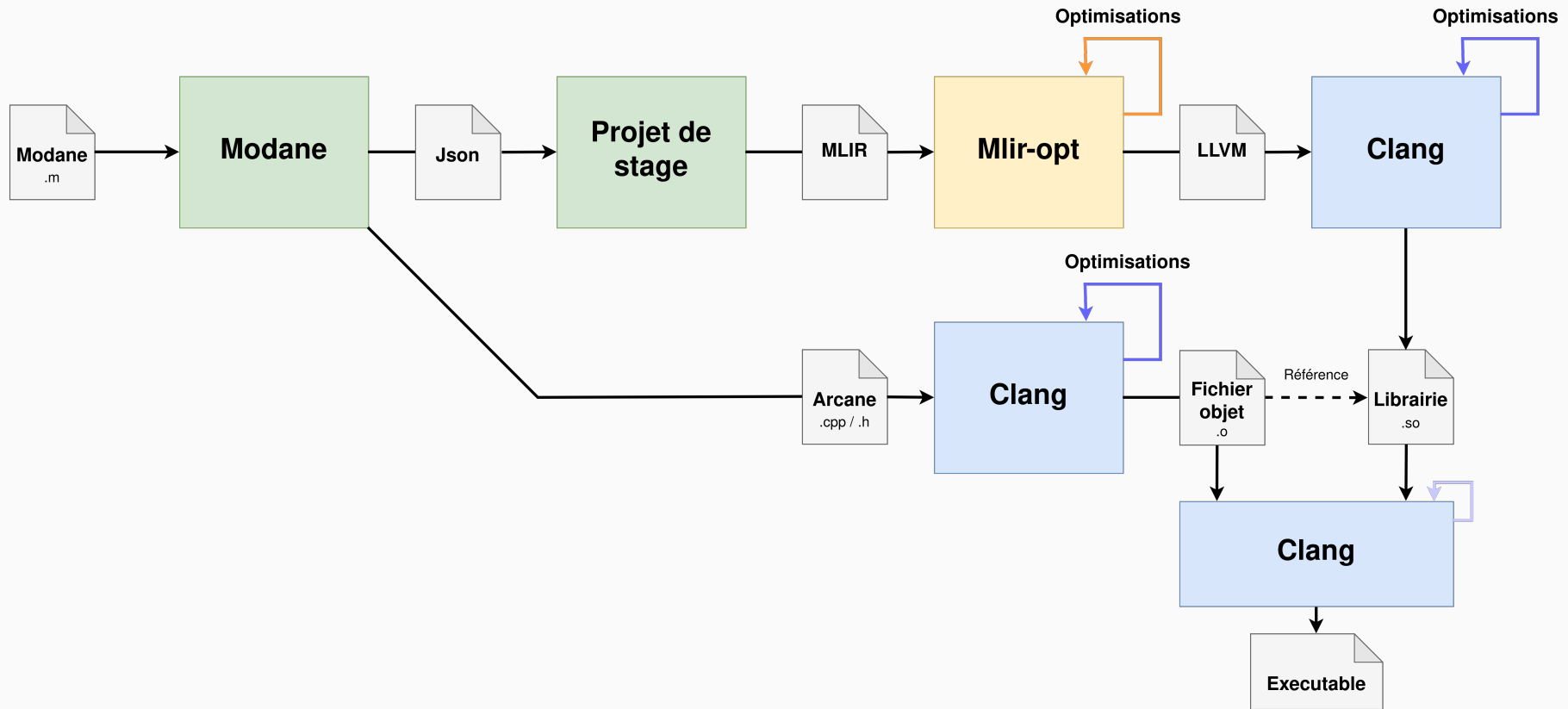


Figure 6: Organisation de la chaine de compilation (honnête)

Description de fonction à générer

```
{ "op": "function",  
  "name": "max",  
  "args": [  
    [ "x", "i64" ],  
    [ "y", "i64" ]  
  ],  
  "body": [  
    { "op": "if",  
      "cond": {  
        "op": "binary", "ope": ">",  
        "lhs": { "name": "x" },  
        "rhs": { "name": "y" }  
      },  
      "then": { "name": "x" },  
      "else": { "name": "y" },  
    },  
  ],  
}
```

Listing 7: AST en Json

Description de fonction à générer

```
{ "op": "function",  
  "name": "max",  
  "args": [  
    [ "x", "i64" ],  
    [ "y", "i64" ]  
  ],  
  "body": [  
    { "op": "if",  
      "cond": {  
        "op": "binary", "ope": ">",  
        "lhs": { "name": "x" },  
        "rhs": { "name": "y" }  
      },  
      "then": { "name": "x" },  
      "else": { "name": "y" },  
    },  
  ],  
}
```

Listing 8: AST en Json

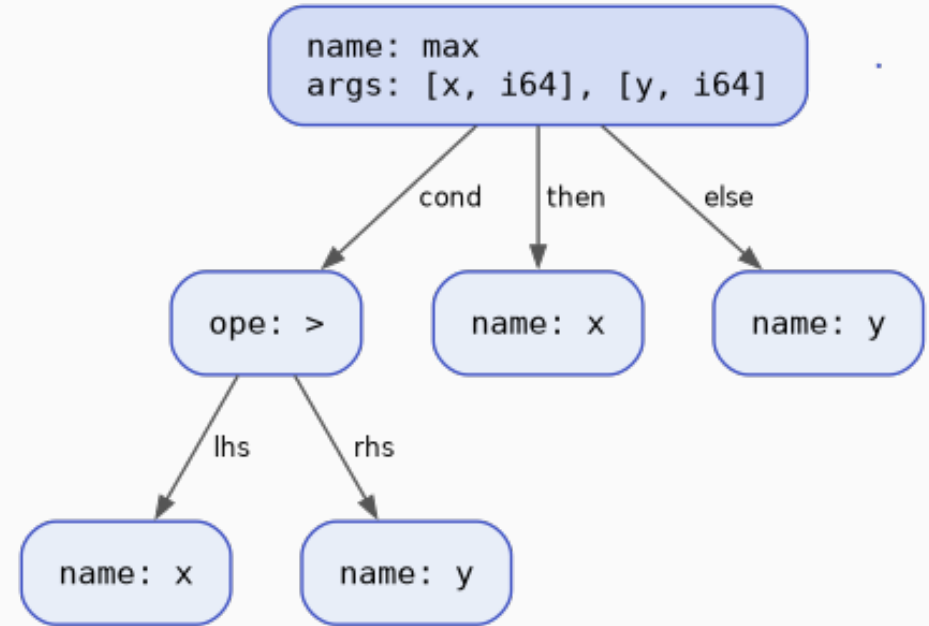


Figure 7: AST composé de structures python

```
class BinaryOp(OpNode):  
    op: Literal["binary"]  
    = "binary"  
    lhs: BaseValue  
    rhs: BaseValue  
    ope: OperatorOp  
  
    def codegen(  
        self, builder: Builder  
    ) -> Sequence[ValNode]:  
        ...
```

Listing 9: Définition d'un
noeud


```
class BinaryOp(OpNode):
    op: Literal["binary"]
    = "binary"
    lhs: BaseValue
    rhs: BaseValue
    ope: OperatorOp

    def codegen(
        self, builder: Builder
    ) -> Sequence[ValNode]:
        ...
```

Listing 11: Définition d'un noeud

```
def codegen(self, builder: Builder) -> Sequence[ValNode]:
    # Codegen recursif
    lhs = self.lhs.codegen(builder) # Noeud
    rhs = self.rhs.codegen(builder)

    # On récupère les valeurs SSA
    ssa_l = lhs.get_SSA()          # Valeur SSA
    ssa_r = rhs.get_SSA()

    # Génération de l'opération
    match self.ope.value:
        case "+":
            op = AddiOp(ssa_l, ssa_r)
            builder.insert(op)      # Opération xDSL
            ssa_result = op.result  # Valeur SSA

    # On renvoie le résultat
    return ValSSA(ssa_result)      # Noeud
```

Listing 12: Implémentation de codegen()

- Votre compilateur ne peut pas faire plus que ce que vous lui donnez
- Nous avons besoin de plusieurs niveaux d'expressivité
- Pensez à ré-utiliser les briques existantes
- Faire son propre langage de programmation: c'est fun et atteignable !

Merci pour votre attention !
Avez-vous des questions ?